Contents

DSA Solutions Guide - Part 2: Linked Lists, Stacks & Queues **3**
Complete JavaScript Solutions with Detailed Explanations . . . 3

LINKED LISTS **3**
Understanding Linked Lists . . . 3

1. Reverse Linked List **3**
Problem Statement . . . 3
Example . . . 3
Approach 1: Iterative - O(n) . . . 4
Explanation . . . 4
Visual Walkthrough . . . 4
Approach 2: Recursive - O(n) . . . 4
Explanation . . . 4

2. Merge Two Sorted Lists **5**
Problem Statement . . . 5
Example . . . 5
Iterative Approach - O(n + m) . . . 5
Recursive Approach . . . 5

3. Linked List Cycle Detection **6**
Problem Statement . . . 6
Floyd's Cycle Detection (Tortoise and Hare) - O(n) . . . 6
Explanation . . . 6
Find Cycle Start Node . . . 6
Why Phase 2 Works (Mathematical Proof) . . . 7

4. Remove Nth Node From End **7**
Problem Statement . . . 7
Example . . . 7
Two Pointers Approach - O(n) . . . 7
Explanation . . . 7

5. Middle of Linked List **8**
Problem Statement . . . 8
Fast and Slow Pointers - O(n) . . . 8
Visualization . . . 8

6. Palindrome Linked List **9**
Problem Statement . . . 9
Approach: Reverse Second Half - O(n) Time, O(1) Space . . . 9

7. Add Two Numbers **10**
Problem Statement . . . 10
Example . . . 10
Solution - O(max(m, n)) . . . 10

8. Sort List (Merge Sort) **10**
Problem Statement . . . 10
Merge Sort Approach . . . 11

9. LRU Cache **11**
Problem Statement . . . 11
Approach: HashMap + Doubly Linked List . . . 12
Explanation . . . 12

10. Merge K Sorted Lists **13**
Problem Statement . . . 13
Approach 1: Min Heap - O(n log k) . . . 13
Approach 2: Divide and Conquer - O(n log k) . . . 15

STACKS **15**
Understanding Stacks . . . 15

11. Valid Parentheses **15**
Problem Statement . . . 15
Example . . . 15
Stack Approach - O(n) . . . 16

12. Min Stack **16**
Problem Statement . . . 16
Approach: Two Stacks . . . 16

13. Daily Temperatures **17**
Problem Statement . . . 17
Example . . . 17
Monotonic Stack - O(n) . . . 17
Explanation . . . 17

14. Decode String **18**
Problem Statement . . . 18
Stack Approach - O(n) . . . 18

15. Largest Rectangle in Histogram **18**
Problem Statement . . . 18
Example . . . 18
Monotonic Stack - O(n) . . . 19
Explanation . . . 19
Visual Walkthrough . . . 19

QUEUES **19**
Understanding Queues . . . 19

16. Implement Queue using Stacks **20**
Problem Statement . . . 20

17. Design Circular Queue **21**

18. Evaluate Reverse Polish Notation **22**
Problem Statement . . . 22
Example . . . 22

19. Basic Calculator II **23**
Problem Statement . . . 23
Example . . . 23

20. Next Greater Element **24**
Problem Statement . . . 24
Monotonic Stack - O(n + m) . . . 24

Summary: Key Patterns **24**
Linked List Patterns . . . 24
Stack/Queue Patterns . . . 24

DSA Solutions Guide - Part 2: Linked Lists, Stacks & Queues

Complete JavaScript Solutions with Detailed Explanations

LINKED LISTS

Understanding Linked Lists

A linked list is a linear data structure where elements are stored in nodes, and each node points to the next node.

```javascript
// Node structure
class ListNode {
    constructor(val = 0, next = null) {
        this.val = val;
        this.next = next;
    }
}

// Helper: Create linked list from array
function createLinkedList(arr) {
    if (arr.length === 0) return null;
    const head = new ListNode(arr[0]);
    let current = head;
    for (let i = 1; i < arr.length; i++) {
        current.next = new ListNode(arr[i]);
        current = current.next;
    }
    return head;
}

// Helper: Convert linked list to array (for visualization)
function linkedListToArray(head) {
    const result = [];
    while (head) {
        result.push(head.val);
        head = head.next;
    }
    return result;
}
```

---

1. Reverse Linked List

Problem Statement

Reverse a singly linked list.

Example

```
Input: 1 -> 2 -> 3 -> 4 -> 5 -> null
Output: 5 -> 4 -> 3 -> 2 -> 1 -> null
```

Approach 1: Iterative - O(n)

Explanation

Use three pointers: prev, current, next. At each step: 1. Save the next node 2. Reverse the current pointer 3. Move prev and current forward

Visual Walkthrough

```
Initial: null <- prev   1 -> 2 -> 3 -> null
                        ^curr

Step 1:  null <- 1      2 -> 3 -> null
         prev    ^curr

Step 2:  null <- 1 <- 2      3 -> null
                 prev ^curr

Step 3:  null <- 1 <- 2 <- 3
                       prev ^curr (null)

Return prev (points to 3)
```

```javascript
function reverseList(head) {
    let prev = null;
    let current = head;

    while (current !== null) {
        // Save next node before we overwrite current.next
        const nextTemp = current.next;

        // Reverse the pointer
        current.next = prev;

        // Move prev and current forward
        prev = current;
        current = nextTemp;
    }

    return prev;  // New head
}

// Time: O(n) - visit each node once
// Space: O(1) - only three pointers
```

Approach 2: Recursive - O(n)

Explanation

The recursive approach works from the end. When we reach the last node, it becomes the new head. As we return, we reverse each pointer.

```javascript
function reverseListRecursive(head) {
    // Base case: empty list or single node
    if (head === null || head.next === null) {
        return head;
    }

    // Recursively reverse the rest
```

```
    const newHead = reverseListRecursive(head.next);

    // Reverse the pointer
    head.next.next = head;  // Next node points back to current
    head.next = null;       // Current points to null (will be fixed by caller)

    return newHead;
}

// Time: O(n) - visit each node once
// Space: O(n) - recursion stack
```

---

2. Merge Two Sorted Lists

Problem Statement

Merge two sorted linked lists into one sorted list.

Example

```
Input: list1 = 1 -> 2 -> 4, list2 = 1 -> 3 -> 4
Output: 1 -> 1 -> 2 -> 3 -> 4 -> 4
```

Iterative Approach - O(n + m)

```
function mergeTwoLists(list1, list2) {
    // Dummy node simplifies edge cases
    const dummy = new ListNode(-1);
    let current = dummy;

    while (list1 !== null && list2 !== null) {
        if (list1.val <= list2.val) {
            current.next = list1;
            list1 = list1.next;
        } else {
            current.next = list2;
            list2 = list2.next;
        }
        current = current.next;
    }

    // Attach remaining nodes
    current.next = list1 !== null ? list1 : list2;

    return dummy.next;
}

// Time: O(n + m)
// Space: O(1) - reusing existing nodes
```

Recursive Approach

```
function mergeTwoListsRecursive(list1, list2) {
    if (list1 === null) return list2;
    if (list2 === null) return list1;
```

```
    if (list1.val <= list2.val) {
        list1.next = mergeTwoListsRecursive(list1.next, list2);
        return list1;
    } else {
        list2.next = mergeTwoListsRecursive(list1, list2.next);
        return list2;
    }
}

// Time: O(n + m)
// Space: O(n + m) - recursion stack
```

---

3. Linked List Cycle Detection

Problem Statement

Detect if a linked list has a cycle (a node's next points to a previous node).

Floyd's Cycle Detection (Tortoise and Hare) - O(n)

Explanation

Use two pointers moving at different speeds. If there's a cycle, fast will eventually catch up to slow. If no cycle, fast will reach null.

```
function hasCycle(head) {
    if (head === null || head.next === null) {
        return false;
    }

    let slow = head;
    let fast = head;

    while (fast !== null && fast.next !== null) {
        slow = slow.next;        // Move 1 step
        fast = fast.next.next;   // Move 2 steps

        if (slow === fast) {
            return true;  // Cycle detected!
        }
    }

    return false;  // Fast reached end, no cycle
}

// Time: O(n)
// Space: O(1)
```

Find Cycle Start Node

```
function detectCycle(head) {
    if (head === null || head.next === null) {
        return null;
    }
```

```
    let slow = head;
    let fast = head;

    // Phase 1: Detect cycle
    while (fast !== null && fast.next !== null) {
        slow = slow.next;
        fast = fast.next.next;

        if (slow === fast) {
            // Phase 2: Find cycle start
            // Move slow to head, keep fast at meeting point
            slow = head;
            while (slow !== fast) {
                slow = slow.next;
                fast = fast.next;
            }
            return slow;  // Cycle start
        }
    }

    return null;  // No cycle
}
```

Why Phase 2 Works (Mathematical Proof)

- Let distance from head to cycle start = a
- Let distance from cycle start to meeting point = b
- Let cycle length = c

When they meet: - Slow traveled: a + b - Fast traveled: a + b + nc (n complete cycles) - Fast travels 2x slow: 2(a + b) = a + b + nc - Therefore: a + b = nc, so a = nc - b = (n-1)c + (c - b)

This means: distance from head to cycle start = distance from meeting point to cycle start (going around the cycle).

---

4. Remove Nth Node From End

Problem Statement

Remove the nth node from the end of the list in one pass.

Example

```
Input: 1 -> 2 -> 3 -> 4 -> 5, n = 2
Output: 1 -> 2 -> 3 -> 5 (removed 4, which is 2nd from end)
```

Two Pointers Approach - O(n)

Explanation

1. Move fast pointer n steps ahead
2. Move both pointers until fast reaches end
3. Now slow is at the node before the one to remove

```
function removeNthFromEnd(head, n) {
    // Dummy node handles edge case of removing first node
    const dummy = new ListNode(0, head);
    let slow = dummy;
```

```
    let fast = dummy;

    // Move fast n+1 steps ahead
    for (let i = 0; i <= n; i++) {
        fast = fast.next;
    }

    // Move both until fast reaches end
    while (fast !== null) {
        slow = slow.next;
        fast = fast.next;
    }

    // Remove the nth node
    slow.next = slow.next.next;

    return dummy.next;
}

// Time: O(n) - single pass
// Space: O(1)
```

---

5. Middle of Linked List

Problem Statement

Return the middle node. If two middle nodes, return the second.

Fast and Slow Pointers - O(n)

```
function middleNode(head) {
    let slow = head;
    let fast = head;

    while (fast !== null && fast.next !== null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow;
}

// Time: O(n)
// Space: O(1)
```

Visualization

```
1 -> 2 -> 3 -> 4 -> 5

Step 0: slow=1, fast=1
Step 1: slow=2, fast=3
Step 2: slow=3, fast=5
Step 3: fast.next=null, stop

Return slow (node 3)
```

6. Palindrome Linked List

Problem Statement

Check if a linked list is a palindrome.

Approach: Reverse Second Half - O(n) Time, O(1) Space

```
function isPalindrome(head) {
    if (head === null || head.next === null) {
        return true;
    }

    // Step 1: Find middle
    let slow = head;
    let fast = head;
    while (fast.next !== null && fast.next.next !== null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    // Step 2: Reverse second half
    let secondHalf = reverseList(slow.next);

    // Step 3: Compare first and second half
    let firstHalf = head;
    let result = true;
    while (secondHalf !== null) {
        if (firstHalf.val !== secondHalf.val) {
            result = false;
            break;
        }
        firstHalf = firstHalf.next;
        secondHalf = secondHalf.next;
    }

    // Step 4: Restore list (optional but good practice)
    slow.next = reverseList(slow.next);

    return result;
}

function reverseList(head) {
    let prev = null;
    while (head !== null) {
        const next = head.next;
        head.next = prev;
        prev = head;
        head = next;
    }
    return prev;
}

// Time: O(n)
```

```
// Space: O(1)
```

---

7. Add Two Numbers

Problem Statement

Add two numbers represented as linked lists (digits in reverse order).

Example

```
Input: l1 = 2 -> 4 -> 3 (represents 342)
       l2 = 5 -> 6 -> 4 (represents 465)
Output: 7 -> 0 -> 8 (represents 807)
342 + 465 = 807
```

Solution - O(max(m, n))

```javascript
function addTwoNumbers(l1, l2) {
    const dummy = new ListNode(0);
    let current = dummy;
    let carry = 0;

    while (l1 !== null || l2 !== null || carry > 0) {
        // Get values (0 if list ended)
        const val1 = l1 !== null ? l1.val : 0;
        const val2 = l2 !== null ? l2.val : 0;

        // Calculate sum and carry
        const sum = val1 + val2 + carry;
        carry = Math.floor(sum / 10);

        // Create new node with digit
        current.next = new ListNode(sum % 10);
        current = current.next;

        // Move to next nodes
        if (l1 !== null) l1 = l1.next;
        if (l2 !== null) l2 = l2.next;
    }

    return dummy.next;
}

// Time: O(max(m, n))
// Space: O(max(m, n)) for result
```

---

8. Sort List (Merge Sort)

Problem Statement

Sort a linked list in O(n log n) time and O(1) space.

Merge Sort Approach

```
function sortList(head) {
    // Base case: empty or single node
    if (head === null || head.next === null) {
        return head;
    }

    // Step 1: Find middle and split
    let slow = head;
    let fast = head;
    let prev = null;

    while (fast !== null && fast.next !== null) {
        prev = slow;
        slow = slow.next;
        fast = fast.next.next;
    }
    prev.next = null;  // Split the list

    // Step 2: Recursively sort both halves
    const left = sortList(head);
    const right = sortList(slow);

    // Step 3: Merge sorted halves
    return mergeTwoLists(left, right);
}

function mergeTwoLists(l1, l2) {
    const dummy = new ListNode(0);
    let current = dummy;

    while (l1 !== null && l2 !== null) {
        if (l1.val <= l2.val) {
            current.next = l1;
            l1 = l1.next;
        } else {
            current.next = l2;
            l2 = l2.next;
        }
        current = current.next;
    }

    current.next = l1 !== null ? l1 : l2;
    return dummy.next;
}

// Time: O(n log n)
// Space: O(log n) for recursion stack
```

---

9. LRU Cache

Problem Statement

Design a Least Recently Used (LRU) cache with O(1) get and put operations.

Approach: HashMap + Doubly Linked List

Explanation

- HashMap: O(1) access to any cache entry
- Doubly Linked List: O(1) removal and insertion for LRU tracking
- Most recently used at head, least recently used at tail

```
class LRUNode {
    constructor(key, val) {
        this.key = key;
        this.val = val;
        this.prev = null;
        this.next = null;
    }
}

class LRUCache {
    constructor(capacity) {
        this.capacity = capacity;
        this.cache = new Map();  // key -> node

        // Dummy head and tail for easier manipulation
        this.head = new LRUNode(0, 0);
        this.tail = new LRUNode(0, 0);
        this.head.next = this.tail;
        this.tail.prev = this.head;
    }

    // Remove node from current position
    _remove(node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    // Add node right after head (most recent)
    _addToHead(node) {
        node.next = this.head.next;
        node.prev = this.head;
        this.head.next.prev = node;
        this.head.next = node;
    }

    get(key) {
        if (!this.cache.has(key)) {
            return -1;
        }

        const node = this.cache.get(key);

        // Move to head (most recently used)
        this._remove(node);
        this._addToHead(node);

        return node.val;
    }

    put(key, value) {
```

```
        if (this.cache.has(key)) {
            // Update existing
            const node = this.cache.get(key);
            node.val = value;
            this._remove(node);
            this._addToHead(node);
        } else {
            // Add new
            const newNode = new LRUNode(key, value);
            this.cache.set(key, newNode);
            this._addToHead(newNode);

            // Evict if over capacity
            if (this.cache.size > this.capacity) {
                const lru = this.tail.prev;
                this._remove(lru);
                this.cache.delete(lru.key);
            }
        }
    }
}

// Time: O(1) for both get and put
// Space: O(capacity)
```

---

10. Merge K Sorted Lists

Problem Statement

Merge k sorted linked lists into one sorted list.

Approach 1: Min Heap - O(n log k)

```
class MinHeap {
    constructor() {
        this.heap = [];
    }

    push(node) {
        this.heap.push(node);
        this._bubbleUp(this.heap.length - 1);
    }

    pop() {
        if (this.heap.length === 0) return null;
        const min = this.heap[0];
        const last = this.heap.pop();
        if (this.heap.length > 0) {
            this.heap[0] = last;
            this._bubbleDown(0);
        }
        return min;
    }

    isEmpty() {
```

```
        return this.heap.length === 0;
    }

    _bubbleUp(index) {
        while (index > 0) {
            const parent = Math.floor((index - 1) / 2);
            if (this.heap[parent].val <= this.heap[index].val) break;
            [this.heap[parent], this.heap[index]] = [this.heap[index], this.heap[parent]];
            index = parent;
        }
    }

    _bubbleDown(index) {
        const length = this.heap.length;
        while (true) {
            let smallest = index;
            const left = 2 * index + 1;
            const right = 2 * index + 2;

            if (left < length && this.heap[left].val < this.heap[smallest].val) {
                smallest = left;
            }
            if (right < length && this.heap[right].val < this.heap[smallest].val) {
                smallest = right;
            }
            if (smallest === index) break;

            [this.heap[smallest], this.heap[index]] = [this.heap[index], this.heap[smallest]];
            index = smallest;
        }
    }
}

function mergeKLists(lists) {
    const heap = new MinHeap();

    // Add first node from each list
    for (const list of lists) {
        if (list !== null) {
            heap.push(list);
        }
    }

    const dummy = new ListNode(0);
    let current = dummy;

    while (!heap.isEmpty()) {
        const node = heap.pop();
        current.next = node;
        current = current.next;

        if (node.next !== null) {
            heap.push(node.next);
        }
    }
```

```
    return dummy.next;
}

// Time: O(n log k) where n = total nodes, k = number of lists
// Space: O(k) for heap
```

Approach 2: Divide and Conquer - O(n log k)

```
function mergeKListsDivideConquer(lists) {
    if (lists.length === 0) return null;

    return mergeRange(lists, 0, lists.length - 1);
}

function mergeRange(lists, start, end) {
    if (start === end) return lists[start];

    const mid = Math.floor((start + end) / 2);
    const left = mergeRange(lists, start, mid);
    const right = mergeRange(lists, mid + 1, end);

    return mergeTwoLists(left, right);
}

// Time: O(n log k)
// Space: O(log k) for recursion
```

STACKS

Understanding Stacks

LIFO (Last In, First Out) data structure.

```
// Using array as stack
const stack = [];
stack.push(1);      // Add to top
stack.push(2);
stack.pop();        // Remove from top -> 2
stack[stack.length - 1];  // Peek top -> 1
```

---

11. Valid Parentheses

Problem Statement

Determine if a string of brackets has valid matching.

Example

```
Input: s = "()[]{}"
Output: true

Input: s = "([)]"
Output: false
```

Stack Approach - O(n)

```
function isValid(s) {
    const stack = [];
    const matching = {
        ')': '(',
        '}': '{',
        ']': '['
    };

    for (const char of s) {
        if (char in matching) {
            // Closing bracket
            if (stack.length === 0 || stack.pop() !== matching[char]) {
                return false;
            }
        } else {
            // Opening bracket
            stack.push(char);
        }
    }

    return stack.length === 0;
}

// Time: O(n)
// Space: O(n)
```

---

12. Min Stack

Problem Statement

Design a stack that supports push, pop, top, and retrieving minimum element in O(1).

Approach: Two Stacks

```
class MinStack {
    constructor() {
        this.stack = [];
        this.minStack = [];  // Stores minimum at each level
    }

    push(val) {
        this.stack.push(val);
        // Push to minStack the minimum so far
        const currentMin = this.minStack.length === 0
            ? val
            : Math.min(val, this.minStack[this.minStack.length - 1]);
        this.minStack.push(currentMin);
    }

    pop() {
        this.stack.pop();
        this.minStack.pop();
    }
```

```
    top() {
        return this.stack[this.stack.length - 1];
    }

    getMin() {
        return this.minStack[this.minStack.length - 1];
    }
}

// All operations: O(1)
// Space: O(n)
```

---

13. Daily Temperatures

Problem Statement

Given temperatures array, return array where answer[i] is number of days until a warmer temperature.

Example

```
Input: temperatures = [73, 74, 75, 71, 69, 72, 76, 73]
Output: [1, 1, 4, 2, 1, 1, 0, 0]
```

Monotonic Stack - O(n)

Explanation

Use a stack to keep track of indices of temperatures we haven't found a warmer day for yet. Stack maintains decreasing temperatures.

```
function dailyTemperatures(temperatures) {
    const n = temperatures.length;
    const result = new Array(n).fill(0);
    const stack = [];  // Stack of indices

    for (let i = 0; i < n; i++) {
        // While current temp is warmer than top of stack
        while (stack.length > 0 && temperatures[i] > temperatures[stack[stack.length - 1]]) {
            const prevIndex = stack.pop();
            result[prevIndex] = i - prevIndex;
        }
        stack.push(i);
    }

    return result;
}

// Time: O(n) - each index pushed and popped at most once
// Space: O(n)
```

---

14. Decode String

Problem Statement

Decode an encoded string like "3[a2[c]]" to "accaccacc".

Stack Approach - O(n)

```
function decodeString(s) {
    const countStack = [];
    const stringStack = [];
    let currentString = '';
    let currentNum = 0;

    for (const char of s) {
        if (char >= '0' && char <= '9') {
            // Build the number
            currentNum = currentNum * 10 + parseInt(char);
        } else if (char === '[') {
            // Save current state and start new
            countStack.push(currentNum);
            stringStack.push(currentString);
            currentNum = 0;
            currentString = '';
        } else if (char === ']') {
            // Pop and repeat
            const repeatCount = countStack.pop();
            const prevString = stringStack.pop();
            currentString = prevString + currentString.repeat(repeatCount);
        } else {
            // Regular character
            currentString += char;
        }
    }

    return currentString;
}

// Time: O(n * maxRepeat)
// Space: O(n)
```

---

15. Largest Rectangle in Histogram

Problem Statement

Find the largest rectangular area in a histogram.

Example

```
Input: heights = [2, 1, 5, 6, 2, 3]
Output: 10 (rectangle of height 5 and width 2)
```

Monotonic Stack - O(n)

Explanation

For each bar, we need to find: - Left boundary: first bar shorter than current on left - Right boundary: first bar shorter than current on right - Area = height * (right - left - 1)

```javascript
function largestRectangleArea(heights) {
    const stack = [];  // Stack of indices, maintaining increasing heights
    let maxArea = 0;

    // Add sentinel to force processing all bars
    heights.push(0);

    for (let i = 0; i < heights.length; i++) {
        // Pop while current is shorter
        while (stack.length > 0 && heights[i] < heights[stack[stack.length - 1]]) {
            const height = heights[stack.pop()];
            // Width extends from previous stack element to current position
            const width = stack.length === 0 ? i : i - stack[stack.length - 1] - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }

    heights.pop();  // Remove sentinel
    return maxArea;
}

// Time: O(n)
// Space: O(n)
```

Visual Walkthrough

```
heights = [2, 1, 5, 6, 2, 3, 0(sentinel)]

i=0: stack=[0]                   (push 2)
i=1: height[1]=1 < height[0]=2  (pop, area=2*1=2)
     stack=[], width=1
     stack=[1]                    (push 1)
i=2: stack=[1,2]                 (push 5)
i=3: stack=[1,2,3]               (push 6)
i=4: height[4]=2 < height[3]=6  (pop, area=6*1=6)
     height[4]=2 < height[2]=5  (pop, area=5*2=10)
     stack=[1,4]                 (push 2)
i=5: stack=[1,4,5]               (push 3)
i=6: height[6]=0 < all           (pop all, calculate areas)

Max area = 10
```

QUEUES

Understanding Queues

FIFO (First In, First Out) data structure.

```
// Using array (inefficient dequeue) or use collections
const queue = [];
queue.push(1);      // Enqueue
queue.shift();      // Dequeue (O(n) for array)

// Better: Use a deque implementation
class Deque {
    constructor() {
        this.items = {};
        this.front = 0;
        this.back = 0;
    }

    pushBack(item) {
        this.items[this.back] = item;
        this.back++;
    }

    popFront() {
        if (this.front === this.back) return undefined;
        const item = this.items[this.front];
        delete this.items[this.front];
        this.front++;
        return item;
    }

    peekFront() {
        return this.items[this.front];
    }

    isEmpty() {
        return this.front === this.back;
    }

    size() {
        return this.back - this.front;
    }
}
```

---

16. Implement Queue using Stacks

Problem Statement

Implement a queue using only two stacks.

```
class MyQueue {
    constructor() {
        this.stackIn = [];   // For push
        this.stackOut = [];  // For pop/peek
    }

    push(x) {
        this.stackIn.push(x);
    }
```

```
    pop() {
        this._moveToOut();
        return this.stackOut.pop();
    }

    peek() {
        this._moveToOut();
        return this.stackOut[this.stackOut.length - 1];
    }

    empty() {
        return this.stackIn.length === 0 && this.stackOut.length === 0;
    }

    _moveToOut() {
        // Only move when stackOut is empty
        if (this.stackOut.length === 0) {
            while (this.stackIn.length > 0) {
                this.stackOut.push(this.stackIn.pop());
            }
        }
    }
}

// Amortized O(1) for all operations
```

---

17. Design Circular Queue

```
class MyCircularQueue {
    constructor(k) {
        this.queue = new Array(k);
        this.size = k;
        this.front = 0;
        this.rear = -1;
        this.count = 0;
    }

    enQueue(value) {
        if (this.isFull()) return false;
        this.rear = (this.rear + 1) % this.size;
        this.queue[this.rear] = value;
        this.count++;
        return true;
    }

    deQueue() {
        if (this.isEmpty()) return false;
        this.front = (this.front + 1) % this.size;
        this.count--;
        return true;
    }

    Front() {
        return this.isEmpty() ? -1 : this.queue[this.front];
```

```
    }

    Rear() {
        return this.isEmpty() ? -1 : this.queue[this.rear];
    }

    isEmpty() {
        return this.count === 0;
    }

    isFull() {
        return this.count === this.size;
    }
}

// All operations: O(1)
```

---

18. Evaluate Reverse Polish Notation

Problem Statement

Evaluate an expression in Reverse Polish Notation (postfix).

Example

```
Input: tokens = ["2", "1", "+", "3", "*"]
Output: 9
Explanation: ((2 + 1) * 3) = 9
```

```
function evalRPN(tokens) {
    const stack = [];
    const operators = new Set(['+', '-', '*', '/']);

    for (const token of tokens) {
        if (operators.has(token)) {
            const b = stack.pop();
            const a = stack.pop();
            let result;

            switch (token) {
                case '+': result = a + b; break;
                case '-': result = a - b; break;
                case '*': result = a * b; break;
                case '/': result = Math.trunc(a / b); break;  // Truncate toward zero
            }

            stack.push(result);
        } else {
            stack.push(parseInt(token));
        }
    }

    return stack[0];
}
```

```
// Time: O(n)
// Space: O(n)
```

---

19. Basic Calculator II

Problem Statement

Implement a calculator for +, -, *, / operations (no parentheses).

Example

```
Input: s = "3+2*2"
Output: 7

Input: s = " 3/2 "
Output: 1
```

```
function calculate(s) {
    const stack = [];
    let num = 0;
    let prevOp = '+';

    for (let i = 0; i < s.length; i++) {
        const char = s[i];

        if (char >= '0' && char <= '9') {
            num = num * 10 + parseInt(char);
        }

        // Process when we hit operator or end of string
        if ((char !== ' ' && !(char >= '0' && char <= '9')) || i === s.length - 1) {
            switch (prevOp) {
                case '+':
                    stack.push(num);
                    break;
                case '-':
                    stack.push(-num);
                    break;
                case '*':
                    stack.push(stack.pop() * num);
                    break;
                case '/':
                    stack.push(Math.trunc(stack.pop() / num));
                    break;
            }
            prevOp = char;
            num = 0;
        }
    }

    // Sum all values in stack
    return stack.reduce((sum, val) => sum + val, 0);
}

// Time: O(n)
```

```
// Space: O(n)
```

---

20. Next Greater Element

Problem Statement

Find the next greater element for each element in nums1 from nums2.

Monotonic Stack - O(n + m)

```
function nextGreaterElement(nums1, nums2) {
    const nextGreater = new Map();
    const stack = [];

    // Build map of next greater elements for nums2
    for (const num of nums2) {
        while (stack.length > 0 && num > stack[stack.length - 1]) {
            nextGreater.set(stack.pop(), num);
        }
        stack.push(num);
    }

    // Look up results for nums1
    return nums1.map(num => nextGreater.get(num) ?? -1);
}

// Time: O(n + m)
// Space: O(m)
```

Summary: Key Patterns

Linked List Patterns

Pattern	When to Use	Examples
Two Pointers (Fast/Slow)	Cycle detection, find middle	Cycle, Middle Node
Dummy Node	Simplify head modifications	Merge, Remove Nth
Reverse	Palindrome, reverse operations	Reverse List
Merge	Combine sorted lists	Merge Two/K Lists

Stack/Queue Patterns

Pattern	When to Use	Examples
Monotonic Stack	Next greater/smaller	Daily Temps, Histogram
Two Stacks	Track min/max, implement queue	Min Stack, Queue
Matching Pairs	Parentheses, expression eval	Valid Parentheses
State Stack	Nested structures	Decode String

Continue to Part 3: Trees & Graphs Solutions